

AROFRAG

Automated Molecular Fragmentation Software

Python & C++ • Automation • Scientific Computing

AROFRAG is a scientific computing software that automates the fragmentation of complex molecules into meaningful substructures for quantum chemistry calculations. It enables systematic workflows, reduces manual work and ensures reproducible, high-throughput computational studies.

1. Molecular Structure Input

Read molecules from various file formats (SD, MOL2, XYZ, etc.)

2. Structure Analysis

Analyze topology, identify rings, branches and key structural motifs

3. Fragmentation Algorithm

Apply AROFRAG rules to generate chemically meaningful fragments

4. Generated Fragments

Systematically produced fragments with properties and metadata

5. Quantum Chemistry Calculations

Export for Gaussian, ORCA or other quantum chemistry packages

High Automation

Reduces manual work and human error

Reproducible

Consistent and traceable workflows

Scalable

Handles large molecules and datasets

Flexible

Custom fragmentation rules and parameters

Key Capabilities

- Graph-based molecular analysis
- Automatic detection of rings, branches and functional groups
- Custom fragmentation rules

Built With

Python

C++

CMake

Fragment validation and filtering

Descriptor calculation for fragments

Batch processing and parallel execution

Export for major quantum chemistry packages

Input Molecule

Structure Analysis

Fragmentation

Generated Fragments

Quantum Calculations

The methodology and software have been validated through peer-reviewed publications in leading journals.

JCTC

JCC

Angewandte Chemie

Project

fragmenter.py

```
1 def fragment_molecule(mol, rules):
2     """Fragment a molecule using AROFRAG rules."""
3     graph = build_graph(mol)
4     cuts = find_cut_bonds(graph, rules)
5     fragments = []
6     for cut in cuts:
7         frags = mol.get_cut(mol, cut)
8         fragments.extend(frags)
9     return fragments
10
11 def apply_cut(mol, cut):
12     """Apply a cut to the molecule and return fragments."""
13     frags = fragment_molecule(mol, rules)
14     return frags
15
16 if __name__ == "__main__":
17     mol = read_molecule("input.sdf")
18     rules = load_rules("config.yaml")
19     frags = fragment_molecule(mol, rules)
20     write_fragments(frags, "output.sdf")
```

AROFRAG - Fragmentation Viewer

Molecule

Frags (22)

Fragment Details

Fragmentation completed in 2.41 s

AI-Powered Virtual Rug & Recommendation System

Python • FastAPI • AI/Computer Vision • Recommendation Engine

1. Customer

• Upload room photo

• Provide preferences

• Browse & interact

2. Web / Frontend

• Room image upload

• Rug visualization

• View recommendations

3. Backend API (FastAPI)

• REST API Endpoints

• Authentication & Validation

• Business Logic

• Request/Response Handling

4. AI & Processing Layer

Image Processing

• Room detection

• Perspective analysis

• Object & surface detection

AI / LLM & Embeddings

• Feature extraction

• LLM integration

• Semantic understanding

5. Recommendation Engine

• Similarity matching

• Style & color matching

• Personalized ranking

• Business rules

6. Product Catalog

• Rugs database

• Attributes & metadata

• Images & details

• Inventory

Deployment Architecture

Users

Ngix Reverse Proxy (HTTPS)

Docker Containerization

FastAPI Application Container

PostgreSQL Database

Redis Cache / Sessions

Object Storage (Images)

Cloud Server (Ubuntu)

Technologies Used

Python

FastAPI

OpenCV

scikit-learn

LangChain

OpenAI API

PostgreSQL

Redis

Docker

Ngix

Smart Visualization

Place rugs in real rooms with accurate perspective and scale.

Personalized Recommendations

AI-driven suggestions based on style, color, room & user preferences.

Scalable & Modular

Clean architecture, easy to extend and integrate new AI models.

Secure & Reliable

HTTPS, validation, and best practices for a production-ready system.

RAG Document Assistant

Ask questions about your documents. Get accurate, grounded answers with sources.

Python • FastAPI • LangChain • Vector Database • LLMs

RAG Pipeline Overview

1. Documents

Upload multiple documents (PDF, DOCX, TXT, etc.)

2. Text Processing & Chunking

Extract text and split into small, meaningful chunks

3. Embeddings Creation

Convert chunks into vector embeddings using an embedding model

4. Vector Database (Storage)

Store embeddings in a vector database for fast similarity search

5. Retrieval (Semantic Search)

Find the most relevant chunks based on the user's question

6. LLM (Generation)

LLM generates an answer using the retrieved context & user question

System Architecture

User

Web UI (React / HTML)

API Gateway

REST API (FastAPI)

RAG Backend Services

Document Loader

Text Splitter

Embeddings (LLM)

Vector Store (OpenAI / FAISS)

Retriever

LLM (OpenAI / GPT)

Response

Answer + Sources

Technologies Used

Python

FastAPI

LangChain

OpenAI API

Qdrant (Vector DB)

Docker

PostgreSQL

& more

What It Enables

Private Q&A on your documents

Accurate, context-aware and up-to-date answers

Secure & fully under your control

Scalable, modular and production-ready

Key Features

- Upload and process multiple document types
- Semantic search with vector similarity
- Grounded answers with source citations
- Chat history and session management
- Easy to extend with different LLMs and models
- RESTful API for easy integration

Ask a question

What are the main benefits of Retrieval-Augmented Generation?

Ask

Answer

Retrieval-Augmented Generation (RAG) combines information retrieval with large language models to generate accurate, context-aware answers based on your documents. The main benefits are:

- More accurate and reliable answers
- Uses your own data (private documents)
- Reduces hallucinations
- Easy to update knowledge base

Sources

- Lewis et al., Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks page 2
- Your_Document.pdf page 5
- FAQ_RAG.pdf page 3

Customer Segmentation & E-Commerce Analytics

Data-driven customer segmentation using machine learning to uncover actionable business insights

Python | Data Science | Machine Learning | E-Commerce Analytics

1. DATA SOURCES (Olist E-Commerce)

Orders

99,441 orders with order details

Customers

96,096 customers with demographics

Order Items

112,650 order items with products & price

Products

32,951 products in catalog

Order Reviews

99,732 reviews

2. END-TO-END MACHINE LEARNING PIPELINE

1. Data Cleaning & EDA

• Handle missing values

• Remove outliers

• Ugly formats

• Exploratory analysis

2. Feature Engineering

• RFM features (Recency, Frequency, Monetary)

• Build feature matrix

• Encode categorical variables

• Order & product aggregations

3. Customer-Level Dataset

• Aggregate by customer

• Build feature matrix

• Encode categorical variables

4. Scaling

• Standardization (StandardScaler)

• Normalization numeric features

5. Clustering

• K-Means clustering

• Elbow method for optimal K

• Train final model

6. Evaluation

• Silhouette Score

• Calinski-Harabasz Index

• Cluster validation

7. Customer Segments & Business Insights

- Interpret segments
- Understand behavior
- Identify opportunities
- Actionable strategies

3. CUSTOMER SEGMENTS (Example)

Segment	Description	Key Characteristics	Size
High-Value Loyal Customers	Best customers, high-value & loyal	• High spend • Frequent purchases • Recent activity	14.2%
Frequent Buyers	Purchase frequently with moderate spend	• High frequency • Medium spend • Active	20.1%
New Customers	Recently joined, low history	• Recent customers • Low frequency • Low spend	24.7%
At-Risk Customers	Were active before, now less engaged	• Low recency • Low frequency • Decreasing spend	18.3%
Low-Engagement Customers	Rare purchases, low activity	• Low frequency • Low spend • Inactive	22.7%

4. VISUALIZATIONS & INSIGHTS

Cluster Visualization (PCA)

Customer Distribution by Cluster

RFM Score Distribution by Cluster

Average Metrics by Cluster

5. BUSINESS RECOMMENDATIONS

- Target high-value loyal customers with VIP programs and exclusive offers.
- Engage new customers with onboarding campaigns to increase repeat purchases.
- Upsell and cross-sell to frequent buyers using personalized recommendations.
- Re-engage at-risk customers with win-back campaigns and discounts.
- Activate low-engagement customers with remarketing and personalized content.

TECHNOLOGIES & TOOLS

Python

pandas

NumPy

scikit-learn

Matplotlib

seaborn

SQL

Git

VS Code

Outcome

Data-driven customer segments and actionable insights that drive better marketing, retention, and growth.